

Installation und Deployment

Entwicklerdokument

Technische Beschreibung für Build, Veröffentlichung und Wartung



Inhaltsverzeichnis

Inhaltsverzeichnis.....	2
1. Zweck des Dokuments	3
2. Entwicklungsumgebung	3
3. Projekt öffnen	3
4. Build-Prozess	4
5. Lokales Ausführen.....	4
6. Veröffentlichung / Deployment	4
7. Deployment-Inhalt	4
8. Datenbank und Dateien	5
9. Technische Architektur	6
10. Deployment-Checkliste	6
11. Häufige technische Fehler.....	7

1. Zweck des Dokuments

Dieses Dokument beschreibt die technische Einrichtung der Entwicklungsumgebung, den Build-Prozess und das Deployment von WPFTowerDefense. Es richtet sich an Entwicklerinnen und Entwickler.

2. Entwicklungsumgebung

Komponente	Beschreibung
IDE	Visual Studio 2022 oder eine kompatible .NET-IDE.
SDK	.NET 8 SDK mit Windows Desktop Workload.
Framework	net8.0-windows mit UseWPF=true.
Datenbank	SQLite-Datei unter WPFTowerDefense/Database/TD.db.
NuGet-Pakete	Microsoft.EntityFrameworkCore.Sqlite 8.0.22 und Prism.Events 9.0.537.

3. Projekt öffnen

Das Projekt wird über die Solution-Datei geöffnet:

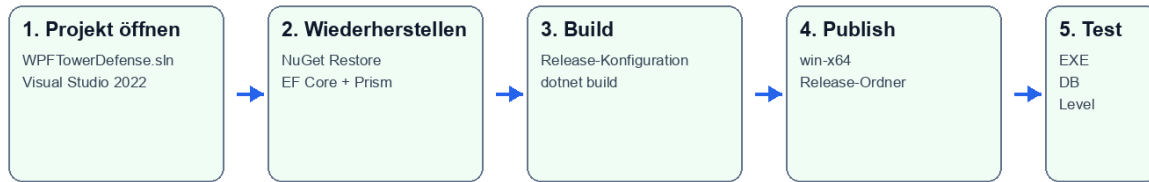
E:\WpfTD2\WPFTowerDefense.sln

Alternativ kann direkt die Projektdatei verwendet werden:

E:\WpfTD2\WPFTowerDefense\WPFTowerDefense.csproj

4. Build-Prozess

Build- und Deployment-Ablauf für Entwickler



Deployment-Inhalt: WPF TowerDefense.exe, DLLs, Database/TD.db, Levels/*.json und eingebettete WPF-Ressourcen.

5. Lokales Ausführen

Zum lokalen Testen kann das Projekt direkt gestartet werden. Beim Start werden die Datenbank Database/TD.db sowie die Leveldateien aus dem Output-Verzeichnis verwendet.

6. Veröffentlichung / Deployment

Für eine Release-Veröffentlichung wird ein Publish-Ordner erzeugt.

7. Deployment-Inhalt

Bestandteil	Muss enthalten sein	Grund
WPF TowerDefense.exe	Ja	Startdatei der Anwendung.
DLL-Dateien	Ja	Benötigte Bibliotheken und NuGet-Abhängigkeiten.
Database/TD.db	Ja	Gegner-, Turm-, Effekt- und Highscore-Daten.
Levels/*.json	Ja	Leveldefinitionen und Gegnerpfade.
Resources und Images	In der Anwendung eingebettet	Grafiken für Oberfläche, Türme, Gegner und Tiles.

8. Datenbank und Dateien

Die Anwendung greift im Code mehrfach über den relativen Pfad Database/TD.db auf die SQLite-Datenbank zu. Deshalb muss die Datenbank im Arbeitsverzeichnis der Anwendung vorhanden sein. Die wichtigsten Tabellen sind EnemyTypes, EnemyLevels, TowerTypes, TowerUpgrades, Effect und Highscores.

Die Leveldateien werden als JSON geladen und müssen im Ordner Levels vorhanden sein.

9. Technische Architektur

Die Anwendung ist nach MVVM aufgebaut. XAML-Views stellen die Oberfläche dar. ViewModels steuern Zustände, Commands und Navigation. Die zentrale Spiellogik liegt im GameManager. WaveGenerator erzeugt Gegnerwellen. Loader-Klassen lesen SQLite- und JSON-Daten und wandeln sie in Datenklassen um.

Bereich	Dateien	Aufgabe
Views	Views/*.xaml	Darstellung der Bildschirme.
ViewModels	ViewModels/*.cs	UI-Zustand, Commands, Navigation.
Spiellogik	GameManager.cs, Enemy.cs, Tower.cs, WaveGenerator.cs	Wellen, Bewegung, Angriffe, Effekte.
Datenzugriff	Database/*.cs	SQLite-Zugriff über Entity Framework Core.
Datenmodelle	GameLogic/*.cs	TowerData, EnemyData, EffectData und LevelData.

10. Deployment-Checkliste

- Release-Build ohne Fehler ausführen.
- Publish-Ordner erzeugen.
- WPFTowerDefense.exe im Publish-Ordner starten.
- Prüfen, ob Database/TD.db im Publish-Ordner vorhanden ist.
- Prüfen, ob Levels/LevelA.json, LevelB.json und LevelC.json vorhanden sind.
- Spiel starten, Level auswählen, erste Welle starten.
- Turmplatzierung, Gegnerwelle, Goldanzeige und Game Over kurz testen.

11. Häufige technische Fehler

Fehler	Ursache	Behebung
FileNotFoundException bei TD.db	Datenbank wurde nicht in den Output kopiert.	csproj CopyToOutputDirectory prüfen und Publish erneut ausführen.
Level kann nicht geladen werden	JSON-Datei fehlt oder ist ungültig.	Levels-Ordner prüfen und JSON-Syntax validieren.
Bild wird nicht angezeigt	Resource-Include fehlt oder Dateiname stimmt nicht.	csproj Resource-Einträge und Groß-/Kleinschreibung prüfen.
SQLite-Fehler	Datenbankstruktur passt nicht zum Loader.	Tabellen und Spalten mit TowerDefenseDbContext und Loadern abgleichen.
Runtime fehlt auf Zielsystem	Framework-dependent Deployment ohne installierte Runtime.	.NET 8 Desktop Runtime installieren oder self-contained publishen.